File Copy

**Institute of Bioinformatics and Applied Biotechnology (IBAB)**

**Internal Assessment 2 - MSc 01 Big Data Biology (Second Semester) April 2024**

**BDBP 207 Machine Learning Laboratory**

**Maximum Marks: 30** **Time:120 min**

**Good Luck !**

**PART - A Answer any two questions** **30 Marks**

**1.** Generate a simulated two-class data set with 100 observations and two features in which there is a visible but non-linear separation between the two classes. Plot the data. Show that in this setting, a support vector machine with a polynomial kernel (with degree greater than 1) or a radial kernel will outperform a support vector classifier on the training data. Which technique performs best on the test data? Make plots and report training and test accuracy in order to back up your assertions. **(15 Marks)**

Hints:

a. Use sklearn.datasets make_moons to generate 100 training and 100 test samples. Use noise = 0.3.

b. Use the below plot_clf() function to draw decision boundaries.

```
def plot_clf(model, df, grid_range, show_contours=False, show_support_vectors=False):
    # Decision boundary plot

    # Get grid of values in given range
    x1 = grid_range
    x2 = grid_range
    xx1, xx2 = np.meshgrid(x1, x2, sparse=False)
    Xgrid = np.stack((xx1.flatten(), xx2.flatten())).T

    # Get decision boundary values for plot grid
    decision_boundary = model.predict(Xgrid)
    decision_boundary_grid = decision_boundary.reshape(len(x2), len(x1))

    # Get decision function values for plot grid
    decision_function = model.decision_function(Xgrid)
    decision_function_grid = decision_function.reshape(len(x2), len(x1))

    fig = plt.figure(figsize=(10, 10))
    if show_contours:
        plt.contourf(x1, x2, decision_function_grid);
    plt.contour(x1, x2, decision_boundary_grid);

    sns.scatterplot(x='x1', y='x2', hue='y', data=df)
    if show_support_vectors:
        sns.scatterplot(x=model.support_vectors_[:, 0], y=model.support_vectors_[:, 1],
color='red', marker='+', s=500)
    plt.show()
```

Interna

**2.** Load NCI60 data from ISLP package. Perform PCA on this data. **(15 Marks)**

```
from ISLP import load_data
NCI60 = load_data('NCI60')
```

a. Plot the first three principal component score vectors (pc1 vs pc2, pc1 vs pc3). Plot the observations (cell lines) corresponding to a given cancer type should be plotted in the same colour.

b. Plot the percent variance explained by the principal components as well as the cumulative percent variance explained.

c. Perform hierarchical clustering (agglomerative clustering) on the first few principal component score vectors and plot the dendrogram. Use 'complete' linkage.

**3.** In this problem, you will perform K-means clustering manually, with K = 2, on a small example with n = 6 observations and d = 2 features. The observations are as follows. **(15 Marks)**

| Observation | X1 | X2 |
|---|---|---|
| 1 | 1 | 4 |
| 2 | 1 | 3 |
| 3 | 0 | 4 |
| 4 | 5 | 1 |
| 5 | 6 | 2 |
| 6 | 4 | 0 |

Write the python code for the following

(a) Plot the observations.
(b) Randomly assign a cluster label to each observation. You can use the np.random.choice() function to do this. Report the cluster labels for each observation.
(c) Compute the centroid for each cluster.
(d) Assign each observation to the centroid to which it is closest, in terms of Euclidean distance. Report the cluster labels for each observation.
(e) Repeat (c) and (d) until the answers obtained stop changing.
(f) In your plot from (a), color the observations according to the cluster labels obtained.